



Secure Shell (SSH): OpenSSH Configuration and Remote Access

Command-Line Configuration, Verification, Key-Based Authentication, and Cross-Platform Access

Submitted By: Muhammad Abdullah

Course Program: DESC Free IT Training of Cloud Computing

by SNSKIES

Class Name: Cloud Computing (Morning)

Assignment Report: SSH Configuration and Remote Access

Instructor: Muhammad Rafid Imran

Date of Submission: August 30, 2026

Table of Contents

Introduction.....	3
01. Task 01: Installing and Verifying OpenSSH.....	4
1. Install OpenSSH Client and Server.....	4
2. Verify Installed SSH Packages.....	4
3. Verify OpenSSH Package Status.....	5
4. Verify SSH Service Status.....	5
5. Inspect sshd_config.....	6
6–9. Network Identity and SSH Connection.....	7
02. Task 02: Access Linux Server Using Windows 11 Command Line.....	9
10. Establish SSH Connection from Windows 11.....	9
11. Verify SSH Service.....	9
03. Task 03: Access an Ubuntu Linux Server from Another Kali Linux Server.....	10
12–17. Install, Enable, Verify, and Connect.....	10
04. Task 04: Install OpenSSH Client on Windows and Configure Key Authentication.....	13
18–20. Install Client, Generate Key, Inspect .ssh.....	13
21–22. Read Public Key and Connect.....	14
23. Configure authorized_keys and Permissions.....	15
24. Test Passwordless SSH Login.....	15
Conclusion.....	16

Introduction

This practical lab explores OpenSSH as the secure remote-access mechanism used to connect to Linux systems from command-line environments. The work covers package installation and verification, SSH service management, server configuration inspection, network identification, remote access from Windows 11, access from Kali Linux, and Ed25519 key-based authentication.

The supplied screenshots are treated as practical evidence. Each screenshot is placed in sequence and is followed by a concise explanation of what is visibly demonstrated. Where a supplied screenshot does not show the exact requested command, the document explicitly identifies the mismatch instead of fabricating evidence.

The practical objectives are to:

- Install and verify the OpenSSH client and server components.
- Confirm that the SSH daemon is running and listening on port 22.
- Identify Linux network addresses used for remote connectivity.
- Establish SSH sessions from Windows 11 and Kali Linux.
- Generate an Ed25519 key pair and configure `authorized_keys` permissions.
- Understand the difference between lab evidence and production hardening.

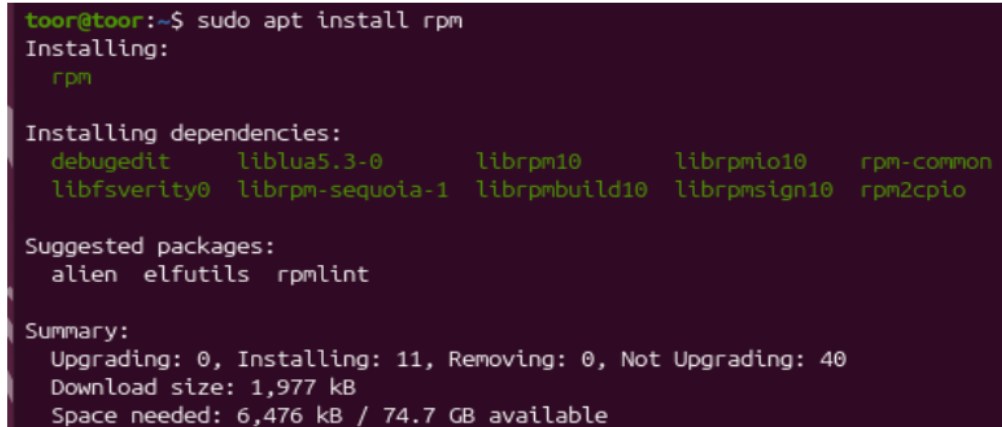
01. Task 01: Installing and Verifying OpenSSH

This task establishes the OpenSSH components, verifies their package state, inspects the server configuration, and confirms the host network identity.

1. Install OpenSSH Client and Server

Command: `sudo apt install openssh-client openssh-server`

Evidence note: The screenshot command and the requested command do not match. It is retained as supplied evidence and is not represented as proof of OpenSSH installation.



```
toor@toor:~$ sudo apt install rpm
Installing:
  rpm

Installing dependencies:
  debugedit      liblua5.3-0      librpm10      librpmio10      rpm-common
  libfsverity0   librpm-sequoia-1 librpmbuild10 librpm-sign10    rpm2cpio

Suggested packages:
  alien elfutils rpmlint

Summary:
  Upgrading: 0, Installing: 11, Removing: 0, Not Upgrading: 40
  Download size: 1,977 kB
  Space needed: 6,476 kB / 74.7 GB available
```

Figure 1. Supplied command-line evidence for Step 1.

What it shows: The supplied screenshot visibly shows an APT installation attempt, but the command displayed is `sudo apt install rpm`, not the requested OpenSSH installation command.

Observed result: APT is installing the `rpm` package and its dependencies. This screenshot therefore does not prove that OpenSSH was installed by this command.

Next step: Use the requested OpenSSH installation command, then verify the package state with `dpkg`.

2. Verify Installed SSH Packages

Command: `dpkg -l | grep ssh`



```
toor@toor:~$ rpm -qa |grep ssh
error: Unable to open sqlite database /var/lib/rpm/rpmdb.sqlite: unable to open database file
error: cannot open Packages index using sqlite - Operation not permitted (1)
error: cannot open Packages database in /var/lib/rpm
toor@toor:~$ dpkg -l | grep ssh
ii  libssh-4:amd64                        0.11.3-1ubuntu2      amd64
tiny C SSH library (OpenSSL flavor)
ii  libssh2-1t64:amd64                  1.11.1-1ubuntu0.26.04.3 amd64
SSH2 client-side library
ii  openssh-client                       1:10.2p1-2ubuntu3.5   amd64
secure shell (SSH) client, for secure access to remote machines
ii  openssh-server                       1:10.2p1-2ubuntu3.5   amd64
secure shell (SSH) server, for secure access from remote machines
ii  openssh-sftp-server                 1:10.2p1-2ubuntu3.5   amd64
secure shell (SSH) sftp server module, for SFTP access from remote machines
ii  ssh-import-id                       5.11-0ubuntu4         all
```

Figure 2. Supplied command-line evidence for Step 2.

What it shows: The terminal displays Debian package records whose names or descriptions contain SSH-related components.

Observed result: OpenSSH-related packages are visible in the package list.

Next step: Inspect the package metadata directly.

3. Verify OpenSSH Package Status

```
Command: dpkg -s openssh-server openssh-client
```

```
toor@toor:~$ dpkg -s openssh-server openssh-client
Package: openssh-server
Status: install ok installed
Priority: optional
Section: net
Installed-Size: 2835
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Architecture: amd64
Multi-Arch: foreign
Source: openssh
Version: 1:10.2p1-2ubuntu3.5
Replaces: openssh-client (<= 1:7.9p1-8), ssh, ssh-krb5
Provides: ssh-server
Depends: libpam-modules, libpam-runtime, lsb-base, openssh-client (= 1:10.2p1-2ubuntu3.5), openssh-sftp-server, procps, ucf, debconf (>= 0.5) | debconf-2.0, systemd | systemd-standalone-sysusers | systemd-sysusers, libaudit1 (>= 1:2.2.1), libc6 (>= 2.42), libcom-err2 (>= 1.43.9), libcrypt1 (>= 1:4.1.0), libg
```

Figure 3. Supplied command-line evidence for Step 3.

What it shows: The package-status query prints metadata for the OpenSSH server package and its relationship with the client package.

Observed result: The screenshot shows **Status: install ok installed** for openssh-server and identifies the OpenSSH package source and dependencies.

Next step: Confirm that the SSH service is operational.

4. Verify SSH Service Status

```
Command: sudo systemctl status ssh
```

```
toor@toor:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enable)
   Active: active (running) since Sun 2026-08-30 11:42:10 PKT; 1h 23min ago
   Invocation: a3b2e61678044dc9af875ba69bd0e303
   TriggeredBy: ● ssh.socket
     Docs: man:sshd(8)
           man:sshd_config(5)
    Main PID: 1919 (sshd)
      Tasks: 1 (limit: 6092)
     Memory: 5.5M (peak: 23M)
        CPU: 162ms
     CGroup: /system.slice/ssh.service
            └─1919 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Aug 30 11:42:10 toor systemd[1]: Starting ssh.service - OpenBSD Secure Shell se
Aug 30 11:42:10 toor sshd[1919]: Server listening on 0.0.0.0 port 22.
Aug 30 11:42:10 toor sshd[1919]: Server listening on :: port 22.
```

Figure 4. Supplied command-line evidence for Step 4.

What it shows: systemd reports the OpenSSH Secure Shell server service state and its runtime details.

Observed result: The screenshot shows **Active: active (running)** and the daemon listening on port 22.

Next step: Inspect the server-side SSH configuration file.

5. Inspect sshd config

```
Command: cat /etc/ssh/sshd_config
```

```
toor@toor:/etc/ssh$ cat sshd_config

# This is the sshd server system-wide configuration file.  See
# sshd_config(5) for more information.

# This sshd was compiled with PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games

# The strategy used for options in the default sshd_config shipped with
# OpenSSH is to specify options with their default value where
# possible, but leave them commented.  Uncommented options override the
# default value.

# In general, the first uncommented definition of an option takes precedence.
# For options that accept multiple values, like 'Port', subsequent definitions
```

Figure 5. Supplied command-line evidence for Step 5.

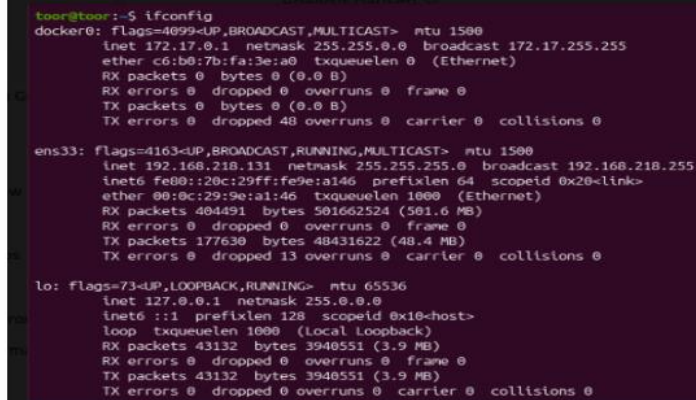
What it shows: The terminal displays the beginning of the system-wide OpenSSH daemon configuration file.

Observed result: The screenshot visibly identifies `/etc/ssh/sshd_config` and shows the standard configuration comments.

Next step: Identify the server's network interfaces and addresses.

6. Display Network Interfaces

Command: `ifconfig`



```
toor@toor:~$ ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0  broadcast 172.17.255.255
    ether c6:b0:7b:fa:3e:a0  txqueuelen 0  (Ethernet)
    RX packets 0  bytes 0 (0.0 B)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 0  bytes 0 (0.0 B)
    TX errors 0  dropped 48 overruns 0  carrier 0  collisions 0

ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 192.168.218.131 netmask 255.255.255.0  broadcast 192.168.218.255
    inet6 fe80::20c:29ff:fe9e:a146 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:9e:a1:46  txqueuelen 1000  (Ethernet)
    RX packets 404491  bytes 501662524 (501.6 MB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 177630  bytes 48431622 (48.4 MB)
    TX errors 0  dropped 13 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000  (Local loopback)
    RX packets 43132  bytes 3940551 (3.9 MB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 43132  bytes 3940551 (3.9 MB)
    TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Figure 6. Supplied command-line evidence for Step 6.

What it shows: The interface listing shows the loopback, Ethernet, and Docker bridge interfaces available on the Ubuntu host.

Observed result: The screenshot includes the host Ethernet interface and the Docker bridge address **172.17.0.1**.

Next step: Inspect addresses using the modern `ip` command.

7. Display IP Address Information

Command: `ip addr show`



```
toor@toor:~$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:9e:a1:46 brd ff:ff:ff:ff:ff:ff
    inet 192.168.218.131/24 brd 192.168.218.255 scope global dynamic noprefixroute ens33
        valid_lft 1415sec preferred_lft 1415sec
    inet6 fe80::20c:29ff:fe9e:a146/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether c6:b0:7b:fa:3e:a0 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
toor@toor:~$ hostname -I
192.168.218.131 172.17.0.1
```

Figure 7. Supplied command-line evidence for Step 7.

What it shows: The command lists interfaces, link state, MAC addresses, IPv4/IPv6 addresses, and address scope.

Observed result: The Ethernet interface is shown with IPv4 address **192.168.218.131/24** and the Docker bridge is shown separately.

Next step: Use `hostname -i` to print host addresses.

8. Display Host IP Addresses

Command: `hostname -i`


```

toor@toor:~$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:9e:a1:46 brd ff:ff:ff:ff:ff:ff
    inet 192.168.218.131/24 brd 192.168.218.255 scope global dynamic noprefixroute ens33
        valid_lft 1415sec preferred_lft 1415sec
    inet6 fe80::20c:29ff:fe9e:a146/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether c6:b0:7b:fa:3e:a0 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
toor@toor:~$ hostname -I
192.168.218.131 172.17.0.1

```

Figure 8. Supplied command-line evidence for Step 8.

What it shows: The supplied screenshot includes hostname -i at the end of the terminal session following the IP address listing.

Observed result: The visible output includes 192.168.218.131 and 172.17.0.1.

Next step: Use the discovered LAN address to establish an SSH session.

9. Connect to the Ubuntu Server

Command: ssh toor@192.168.xxx.xxx

```

toor@toor:~$ ssh toor@192.168.218.131
The authenticity of host '192.168.218.131 (192.168.218.131)' can't be established.
ED25519 key fingerprint is: SHA256:X1KBVby02MtcwSBuuIQpVqXEAi4Wkqygt2jCLCVXCxc
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:2: [hashed name]
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.218.131' (ED25519) to the list of known hosts.
toor@192.168.218.131's password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-30-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

35 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

15 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Sun Aug 30 14:09:26 2026 from 192.168.218.1

```

Figure 9. Supplied command-line evidence for Step 9.

What it shows: The SSH client initiates a connection to the Ubuntu server and performs host-key verification before login.

Observed result: The screenshot shows a successful SSH login to 192.168.218.131 and the Ubuntu welcome banner.

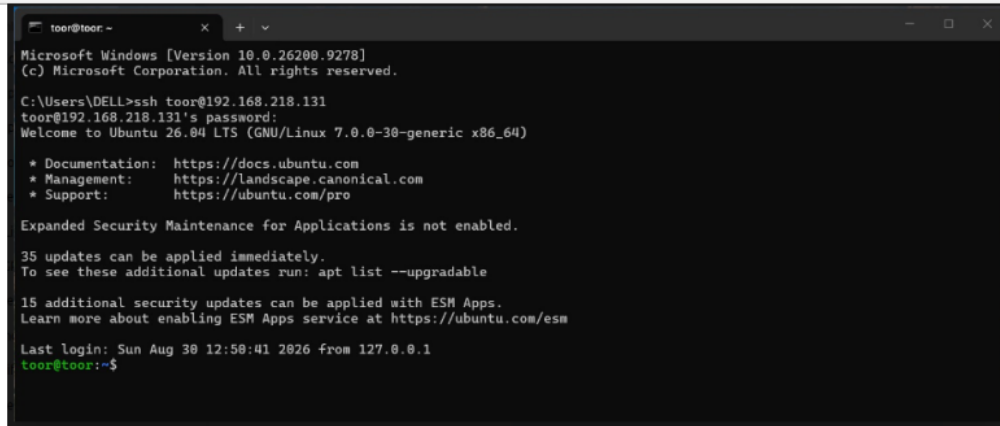
Next step: Repeat the remote-access workflow from Windows 11.

02. Task 02: Access Linux Server Using Windows 11 Command Line

This task demonstrates that a Windows 11 command-line environment can establish an SSH session to the Ubuntu Linux server.

10. Establish SSH Connection from Windows 11

Command: `ssh toor@192.168.xxx.xxx`



```
Microsoft Windows [Version 10.0.26200.9278]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DELL>ssh toor@192.168.218.131
toor@192.168.218.131's password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-30-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

35 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

15 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Sun Aug 30 12:58:41 2026 from 127.0.0.1
toor@toor:~$
```

Figure 10. Supplied command-line evidence for Step 10.

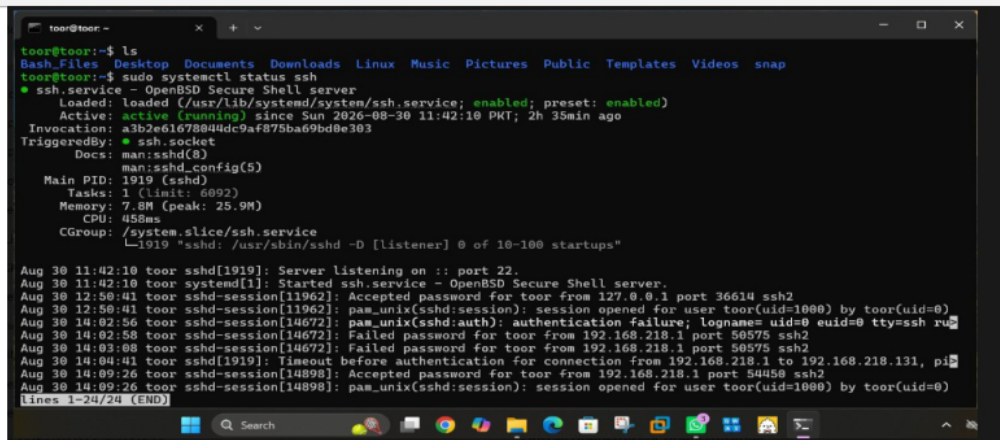
What it shows: Windows Command Prompt runs the OpenSSH client and connects to the Ubuntu host.

Observed result: The screenshot shows the SSH command, password prompt, Ubuntu 26.04 LTS welcome banner, and a Linux shell prompt.

Next step: Verify the SSH daemon from the remote Ubuntu session.

11. Verify SSH Service

Command: `sudo systemctl status ssh`



```
toor@toor:~$ ls
Bash_Files Desktop Documents Downloads Linux Music Pictures Public Templates Videos snap
toor@toor:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-08-30 11:42:10 PKT; 2h 35min ago
     Invocation: a3b2e61678044dc9af875ba69bd0e303
   TriggeredBy: ● ssh.socket
                Docs: man:sshd(8)
                   man:sshd_config(5)
   Main PID: 1919 (sshd)
    Tasks: 1 (limit: 6092)
   Memory: 7.8M (peak: 25.9M)
      CPU: 458ms
   CGroup: /system.slice/ssh.service
           └─1919 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Aug 30 11:42:10 toor sshd[1919]: Server listening on :: port 22.
Aug 30 11:42:10 toor systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
Aug 30 12:58:41 toor sshd-session[11962]: Accepted password for toor from 127.0.0.1 port 36614 ssh2
Aug 30 12:58:41 toor sshd-session[11962]: pam_unix(sshd:session): session opened for user toor(uid=1000) by toor(uid=0)
Aug 30 14:02:56 toor sshd-session[14672]: pam_unix(sshd:auth): authentication failure; logname=uid=0 euid=0 tty=ssh run=
Aug 30 14:02:58 toor sshd-session[14672]: Failed password for toor from 192.168.218.1 port 50575 ssh2
Aug 30 14:03:08 toor sshd-session[14672]: Failed password for toor from 192.168.218.1 port 50575 ssh2
Aug 30 14:04:41 toor sshd[1919]: Timeout before authentication for connection from 192.168.218.1 to 192.168.218.131, pi
Aug 30 14:09:26 toor sshd-session[14898]: Accepted password for toor from 192.168.218.1 port 54050 ssh2
Aug 30 14:09:26 toor sshd-session[14898]: pam_unix(sshd:session): session opened for user toor(uid=1000) by toor(uid=0)
lines 1-24/24 (END)
```

Figure 11. Supplied command-line evidence for Step 11.

What it shows: The remote Ubuntu terminal displays the systemd status of the OpenSSH service.

Observed result: The screenshot shows **active (running)** and the SSH daemon listening on port 22.

Next step: Test access from another Linux distribution.

03. Task 03: Access an Ubuntu Linux Server from Another Kali Linux Server

This task uses Kali Linux as the SSH client and demonstrates remote access to the Ubuntu server over the LAN.

12. Install OpenSSH Client and Server on Kali Linux

Command: `sudo apt update && install -y openssh-server openssh-client`

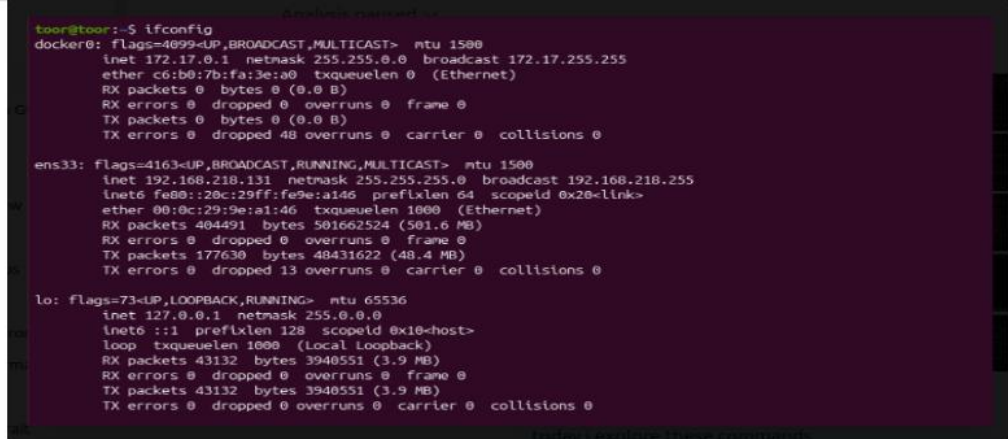
Screenshot evidence: No supplied screenshot shows this exact command. The step is documented without invented evidence.

What it shows: No supplied screenshot in the provided evidence set visibly shows this exact command. It is therefore documented without a fabricated screenshot.

Next step: Verify the Kali network interface configuration.

13. Display Network Interfaces

Command: `ifconfig`



```
toor@toor:~$ ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    ether c6:b0:7b:fa:3e:a0 txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 48 overruns 0 carrier 0 collisions 0

ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.218.131 netmask 255.255.255.0 broadcast 192.168.218.255
    inet6 fe80::20c:29ff:fe9e:a146 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:9e:a1:46 txqueuelen 1000 (Ethernet)
    RX packets 404491 bytes 501662524 (501.6 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 177630 bytes 48431622 (48.4 MB)
    TX errors 0 dropped 13 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 43132 bytes 3940551 (3.9 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 43132 bytes 3940551 (3.9 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Figure 13. Supplied command-line evidence for Step 13.

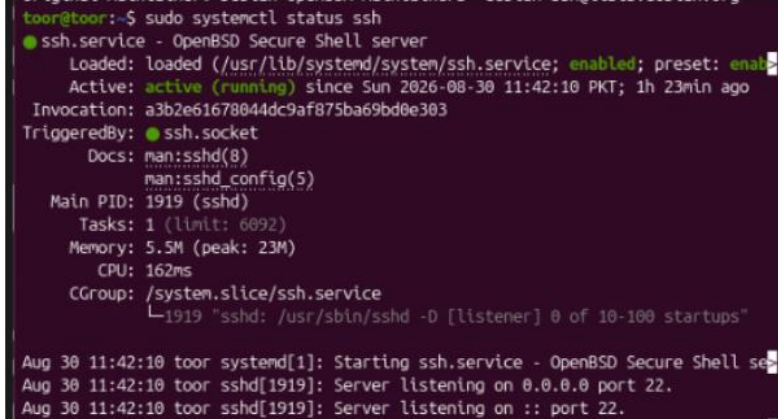
What it shows: The supplied interface listing shows the Linux networking state, including the Ethernet and Docker interfaces.

Observed result: The screenshot visibly includes interface and address information used for connectivity verification.

Next step: Confirm that the SSH service is running.

14. Verify SSH Service

Command: `sudo systemctl status ssh`



```
toor@toor:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enab
   Active: active (running) since Sun 2026-08-30 11:42:10 PKT; 1h 23min ago
   Invocation: a3b2e61678044dc9af875ba69bd0e303
   TriggeredBy: ● ssh.socket
     Docs: man:sshd(8)
           man:sshd_config(5)
    Main PID: 1919 (sshd)
      Tasks: 1 (limit: 6092)
     Memory: 5.5M (peak: 23M)
        CPU: 162ms
     CGroup: /system.slice/ssh.service
            └─1919 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Aug 30 11:42:10 toor systemd[1]: Starting ssh.service - OpenBSD Secure Shell se
Aug 30 11:42:10 toor sshd[1919]: Server listening on 0.0.0.0 port 22.
Aug 30 11:42:10 toor sshd[1919]: Server listening on :: port 22.
```

Figure 14. Supplied command-line evidence for Step 14.

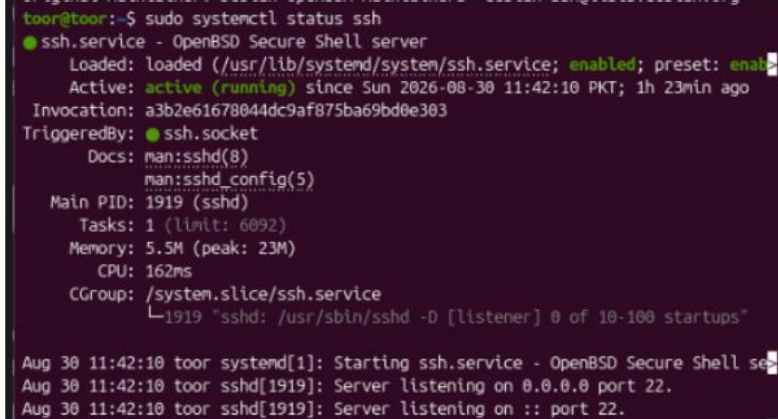
What it shows: systemd reports the OpenSSH Secure Shell server state.

Observed result: The screenshot shows the service in an active/running state and listening on port 22.

Next step: Enable SSH at boot and start it immediately.

15. Enable SSH and Start It Immediately

Command: `sudo systemctl enable --now ssh`



```
toor@toor:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enab
   Active: active (running) since Sun 2026-08-30 11:42:10 PKT; 1h 23min ago
   Invocation: a3b2e61678044dc9af875ba69bd0e303
   TriggeredBy: ● ssh.socket
     Docs: man:sshd(8)
           man:sshd_config(5)
    Main PID: 1919 (sshd)
      Tasks: 1 (limit: 6092)
     Memory: 5.5M (peak: 23M)
        CPU: 162ms
     CGroup: /system.slice/ssh.service
            └─1919 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Aug 30 11:42:10 toor systemd[1]: Starting ssh.service - OpenBSD Secure Shell se
Aug 30 11:42:10 toor sshd[1919]: Server listening on 0.0.0.0 port 22.
Aug 30 11:42:10 toor sshd[1919]: Server listening on :: port 22.
```

Figure 15. Supplied command-line evidence for Step 15.

What it shows: The supplied screenshot shows the resulting SSH service state rather than the enable command itself.

Observed result: The visible service evidence indicates that SSH is running. The screenshot does not independently prove that the enable operation was the command that produced the state.

Next step: Run `systemctl status ssh` again to verify the final state.

16. Verify SSH Service Is Active

Command: `sudo systemctl status ssh`

```
toor@toor:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enable)
   Active: active (running) since Sun 2026-08-30 11:42:10 PKT; 1h 23min ago
     Invocation: a3b2e61678044dc9af875ba69bd0e303
   TriggeredBy: ● ssh.socket
       Docs: man:sshd(8)
             man:sshd_config(5)
    Main PID: 1919 (sshd)
      Tasks: 1 (limit: 6092)
     Memory: 5.5M (peak: 23M)
        CPU: 162ms
       CGroup: /system.slice/ssh.service
              └─1919 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Aug 30 11:42:10 toor systemd[1]: Starting ssh.service - OpenBSD Secure Shell se
Aug 30 11:42:10 toor sshd[1919]: Server listening on 0.0.0.0 port 22.
Aug 30 11:42:10 toor sshd[1919]: Server listening on :: port 22.
```

Figure 16. Supplied command-line evidence for Step 16.

What it shows: The service-status evidence confirms the operational state of the SSH daemon.

Observed result: The screenshot shows **Active: active (running)**.

Next step: Connect from Kali Linux to the Ubuntu server.

17. Connect from Kali Linux to the Ubuntu Server

Command: `ssh toor@192.168.xxx.xxx`

```
(toor@kali)-[~]
└─$ hostname -i
127.0.1.1

(toor@kali)-[~]
└─$ ssh toor@192.168.218.131
The authenticity of host '192.168.218.131 (192.168.218.131)' can't be established
ED25519 key fingerprint is: SHA256:X1K8Vby02MtcwsBuuIQpVqXEA14Wkqygt2JCLCVXcXc
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.218.131' (ED25519) to the list of known hosts
toor@192.168.218.131's password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-30-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

35 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

15 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Sun Aug 30 14:27:15 2026 from 192.168.218.131
toor@toor:~$ ls
Bash_Files Desktop Documents Downloads Linux Music Pictures Public Temp
toor@toor:~$
```

Figure 17. Supplied command-line evidence for Step 17.

What it shows: The Kali Linux terminal launches an SSH session to the Ubuntu host and presents the remote Ubuntu login environment.

Observed result: The screenshot shows Kali Linux initiating SSH and the Ubuntu welcome banner after successful authentication.

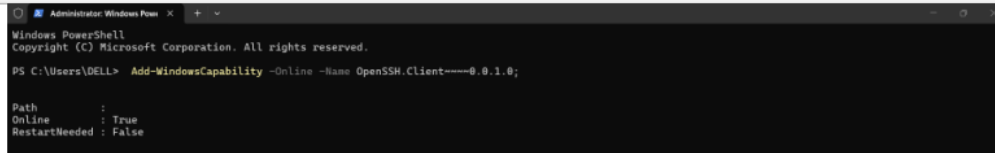
Next step: Configure Windows OpenSSH and key-based authentication.

04. Task 04: Install OpenSSH Client on Windows and Configure Key Authentication

This task installs the Windows OpenSSH client, generates an Ed25519 key pair, transfers the public-key trust to the Linux account, and prepares passwordless SSH access.

18. Install OpenSSH Client on Windows

Command: `Add-WindowsCapability -Online -Name OpenSSH.Client~0.0.1.0;`



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\DELL> Add-WindowsCapability -Online -Name OpenSSH.Client~0.0.1.0;

Path      :
Online    : True
RestartNeeded : False
```

Figure 18. Supplied command-line evidence for Step 18.

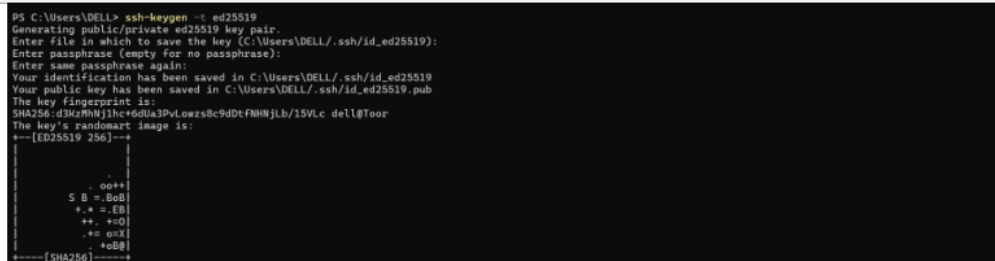
What it shows: Windows PowerShell uses Add-WindowsCapability to add the OpenSSH Client optional Windows capability.

Observed result: The screenshot reports **Online: True** and **RestartNeeded: False**, indicating the capability operation completed without requiring a restart.

Next step: Generate an Ed25519 key pair.

19. Generate Ed25519 SSH Key Pair

Command: `ssh-keygen -t ed25519`



```
PS C:\Users\DELL> ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (C:\Users\DELL\.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\DELL\.ssh/id_ed25519
Your public key has been saved in C:\Users\DELL\.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:d8KrhHj1he+6d0a3P4Lwz8c9d0tFNMJLb/15Vlc dell@Toor
The key's randomart image is:
+--[ED25519 256]--+
|
|      . oo++
|    S B =.BoB|
|  +. +.E|
| ++. +o|
| .+ = o=X|
|  +oB|
+-----[SHA256]-----+
```

Figure 19. Supplied command-line evidence for Step 19.

What it shows: The command creates a modern Ed25519 public/private key pair in the user's SSH directory.

Observed result: The screenshot shows successful key generation and identifies the private key as `C:\Users\DELL\.ssh\id_ed25519` and the public key as `id_ed25519.pub`.

Next step: List the SSH directory to verify the files.

20. List SSH Directory Contents

Command: `ls .ssh`



```
PS C:\Users\DELL> ls .ssh

Directory: C:\Users\DELL\.ssh

Mode                LastWriteTime         Length Name
----                -
-a-----          8/30/2026  6:19 PM           399 id_ed25519
-a-----          8/30/2026  6:19 PM            92 id_ed25519.pub
-a-----          8/30/2026  2:09 PM           846 known_hosts
-a-----          8/30/2026  2:02 PM            98 known_hosts.old
```

Figure 20. Supplied command-line evidence for Step 20.

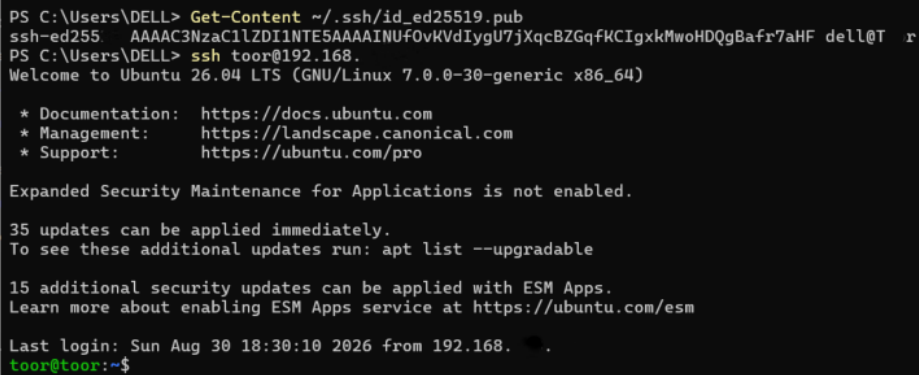
What it shows: PowerShell lists the contents of the user's .ssh directory.

Observed result: The screenshot shows `id_ed25519`, `id_ed25519.pub`, `known_hosts`, and `known_hosts.old`.

Next step: Read the public key for authorized_keys configuration.

21. Read Public Key Content (Copy Output to Clipboard)

Command: Get-Content ~/.ssh/id_ed25519.pub



```
PS C:\Users\DELL> Get-Content ~/.ssh/id_ed25519.pub
ssh-ed255 AAAAC3NzaC1lZDI1NTE5AAAAINUfOvKVdIygU7jXqcBZGqfKCIGxkMwoHDQgBafr7aHF dell@T r
PS C:\Users\DELL> ssh toor@192.168.
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-30-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

35 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

15 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Sun Aug 30 18:30:10 2026 from 192.168. .
toor@toor:~$
```

Figure 21. Supplied command-line evidence for Step 21.

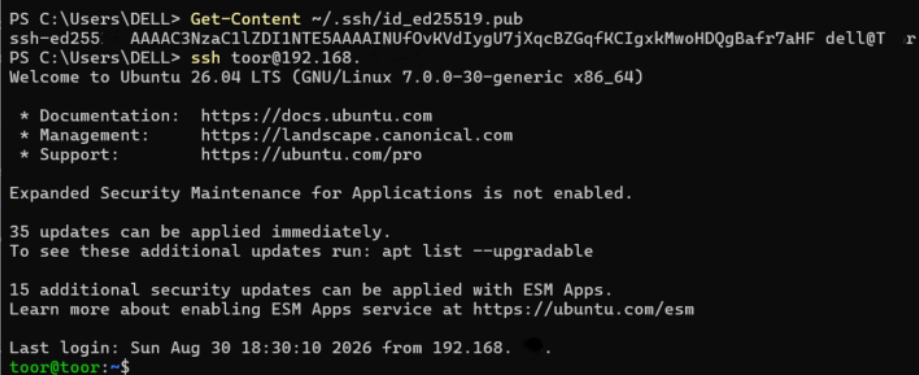
What it shows: PowerShell prints the public Ed25519 key as a single SSH public-key record.

Observed result: The screenshot visibly begins with **ssh-ed25519** and shows the public-key material. The private key is not displayed.

Next step: Use the public key to establish or verify the remote SSH connection.

22. Connect to Remote Machine

Command: ssh toor@192.168.xxx.xxx



```
PS C:\Users\DELL> Get-Content ~/.ssh/id_ed25519.pub
ssh-ed255 AAAAC3NzaC1lZDI1NTE5AAAAINUfOvKVdIygU7jXqcBZGqfKCIGxkMwoHDQgBafr7aHF dell@T r
PS C:\Users\DELL> ssh toor@192.168.
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-30-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

35 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

15 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Sun Aug 30 18:30:10 2026 from 192.168. .
toor@toor:~$
```

Figure 22. Supplied command-line evidence for Step 22.

What it shows: The PowerShell session connects to the Ubuntu server using the OpenSSH client.

Observed result: The screenshot shows the Ubuntu 26.04 LTS banner and a remote **toor@toor** shell prompt. No password prompt is visible in this screenshot.

Next step: Create the remote SSH directory and authorized_keys file.

23. Create SSH Folder and Configure authorized_keys Permissions

```
Command: mkdir -p ~/.ssh
chmod 700 ~/.ssh
nano ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

```
toor@toor:~$ mkdir -p ~/.ssh
chmod 700 ~/.ssh
nano ~/.ssh/authorized_keys
toor@toor:~$ chmod 600 ~/.ssh/authorized_keys
exit
logout
Connection to 192.168.218.131 closed.
toor@toor:~$
```

Figure 23. Supplied command-line evidence for Step 23.

What it shows: The remote Ubuntu terminal shows creation of the .ssh directory, restrictive directory permissions, opening authorized_keys for key insertion, and restrictive file permissions.

Observed result: The screenshot visibly shows `mkdir -p ~/.ssh`, `chmod 700 ~/.ssh`, `nano ~/.ssh/authorized_keys`, and `chmod 600 ~/.ssh/authorized_keys`. It also shows the SSH session being closed.

Next step: Reconnect from Windows and verify that authentication can proceed without a password.

24. Test Passwordless SSH Login

```
Command: ssh toor@192.168.xxx.xxx
```

```
PS C:\Users\DELL> Get-Content ~/.ssh/id_ed25519.pub
ssh-ed255 AAAAC3NzaC1lZDI1NTE5AAAAINUfOvKVdIygU7jXqcBZGqfKCIgxmMwoHDQgBafr7aHF dell@T  r
PS C:\Users\DELL> ssh toor@192.168.
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-30-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

35 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

15 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Sun Aug 30 18:30:10 2026 from 192.168.
toor@toor:~$
```

Figure 24. Supplied command-line evidence for Step 24.

What it shows: The PowerShell screenshot shows an SSH connection to the Ubuntu server without a visible password prompt.

Observed result: The Ubuntu welcome banner is displayed and the remote shell opens directly. This is consistent with key-based authentication, although the screenshot alone does not expose the full server-side authorized_keys contents.

Next step: For production, protect private keys with a strong passphrase and enforce least-privilege SSH configuration.

Conclusion

This practical exercise demonstrates the complete lifecycle of basic SSH access: installing OpenSSH components, verifying package state, checking the SSH daemon, inspecting its configuration, identifying the server's LAN address, and establishing remote sessions from Windows 11 and Kali Linux. The final task extends the workflow to Ed25519 public-key authentication and the correct permissions for the remote SSH directory and `authorized_keys` file.

The supplied screenshots provide direct evidence for most of the workflow. Where an exact screenshot was not available or where a screenshot displayed a different command than the requested one, the document explicitly records that limitation rather than claiming unsupported success. This distinction is important in engineering documentation because observed evidence must remain separate from expected results and recommendations.

For production environments, SSH should be hardened through least-privilege accounts, key-based authentication, protected private keys, appropriate firewall rules, controlled management networks, logging and monitoring, and carefully reviewed `sshd_config` settings. The lab demonstrates the fundamentals; production readiness requires additional security and operational controls.